



# Hosting & Web Services

*“Working outside of local environment...  
How does one achieve this?”*



# INNEHÅLLSFÖRTECKNING

01

Översikt

03

Step by Step  
Database Setup

02

Step by Step  
hosting /w  
MongoDB

04

Eget arbete  
&  
Uppgifter



01

ÖVERSIKT

# Kursplan



## Utbildningsmoment:

- Teoretiska och praktiska övningar ✓
- Webbkommunikation ✓
- Webservices ✓
- HTTP/HTTPS ✓
- JSON
- XML
- Java
- Databaser ✓
- Java applikationer

# Översikt

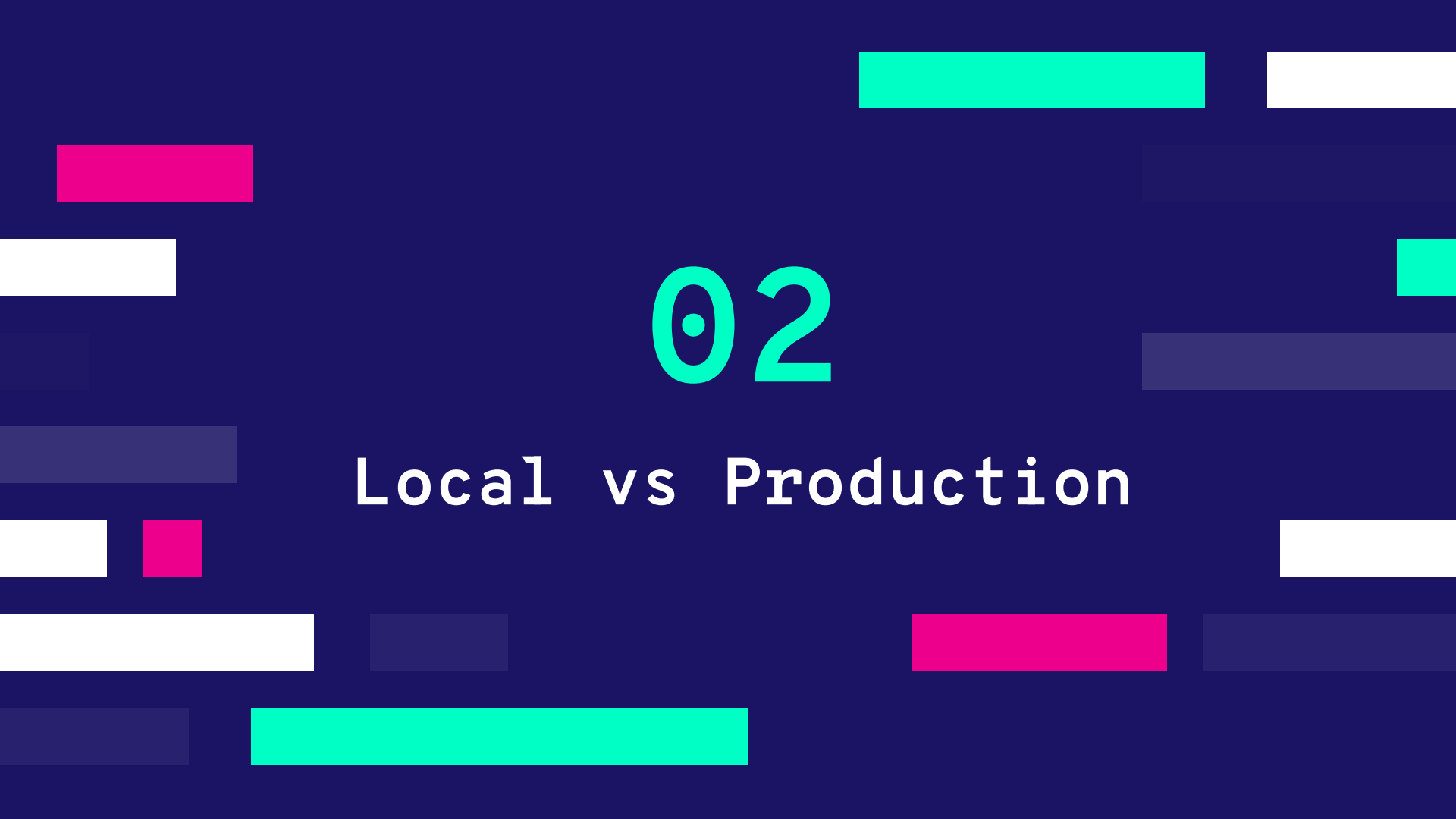


# Översikt

## *Genomgång:*

- Render (Hosting)
- Database (noSQL - example)





02

Local vs Production





**Before we begin..**



```
{
  "compilerOptions": {
    // File Layout
    "rootDir": "./src",
    "outDir": "./dist",

    // Environment Settings
    "module": "NodeNext",
    "target": "ES2022",
    "types": ["node"],

    // Other Outputs
    "sourceMap": true,
    "declaration": true,
    "declarationMap": true,

    // Stricter Typechecking Options
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,

    // Recommended Options
    "strict": true,
    "verbatimModuleSyntax": true,
    "isolatedModules": true,
    "skipLibCheck": true
  },
  "include": ["src"]
}
```



# tsconfig.json

Replace your old content with this

## Why?

We need a 'dist' folder for our 'build'

# Fix port

(Innan vi lanserar, sätt upp PORT)

✓  src

 index.ts

```
const port: number = Number(env.PORT) || 3000 // Could crash
```



 .env

 .env

1 MY\_GLOBAL\_TEST\_SECRET=XvkxYNDsi/7M7FBMlYGm4quoFZBK2Upo+mRv

2 PORT=3000

3

4

Definiera PORT

# Fix Render Domain address

(Innan vi lanserar, sätt upp PORT)

src  
index.ts

```
app.listen(port, "0.0.0.0", () => {  
  console.log(`Listening to port ${port}`)  
})
```



Render

<https://render.com> › docs › web... · Översätt den här sidan

## Web Services

Every **Render** web service must bind to a **port** on host `0.0.0.0` to serve HTTP requests. **Render** forwards inbound requests to your web service at this **port** (it is ...

## ▼ EXAMPLE

>  dist

>  node\_modules

▼  src

 index.ts

 .env

 .gitignore

 env.d.ts

 package-lock.json

 package.json

 tsconfig.json



.gitignore

1 node\_modules/

2 .gitignore

3 .env

4 dist/

Inom din .gitignore

Lägg till **dist/**



Step by step

# Local vs Production

## (Why either?)

### Debugging

- Quicker setup
- Faster Debugging
- Doesn't reflect actual results

### Complete setup

- More error prone
- Slower development
- Reflects results more accurately to production



# Hosting (Render)



**Source:** <https://render.com/>

Free Tier, popular for Node projects, easy setup, quick setup,  
github integration, custom domains, includes https

## Hobby

Current plan

For hobbyists and students

**\$0 USD**

per user/month plus compute costs\*

Select plan →

Deploy full-stack apps in minutes



Fully-managed datastores



Custom domains



Global CDN & regional hosting



Get security out of the box



Email support

**Free Tier**  
(No Credit Card needed)

# Drawbacks

(15 minutes)

Hi there!

We noticed your free web service,  
**WS\_node\_test**, recently spun down after 15  
minutes of inactivity.

# Hacker solution 2.0

(15 minutes problem)

Automatisera request som skickas var 14:e minut



# Setup Account



## Work Lab

Navigera till  
<https://render.com/>

Gör en **'signup'** med **GitHub**  
**Kontot** för **enklast upplevelse**

# New Project

(Render.com)

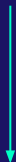
Your fastest  
path to  
production

Build, deploy, and scale your apps with unparalleled ease – from your first user to your billionth

Get Started for Free >

Contact Sales >

# Web Services (New Project)



## Static Sites

Static content served over a global CDN. Ideal for frontend, blogs, and content sites.

[New Static Site →](#)

## Web Services

Dynamic web app. Ideal for full-stack apps, API servers, and mobile backends.

[New Web Service →](#)

## Private Services

Web app hosted on a private network, accessible only from your other Render services.

[New Private Service →](#)

## Background Workers

Long-lived services that process async tasks, usually from a job queue.

[New Worker →](#)

## Cron Jobs

Short-lived tasks that run on a periodic schedule.

[New Cron Job →](#)

## Postgres

Relational data storage. Supports point-in-time recovery, read replicas, and high availability.

[New Postgres →](#)

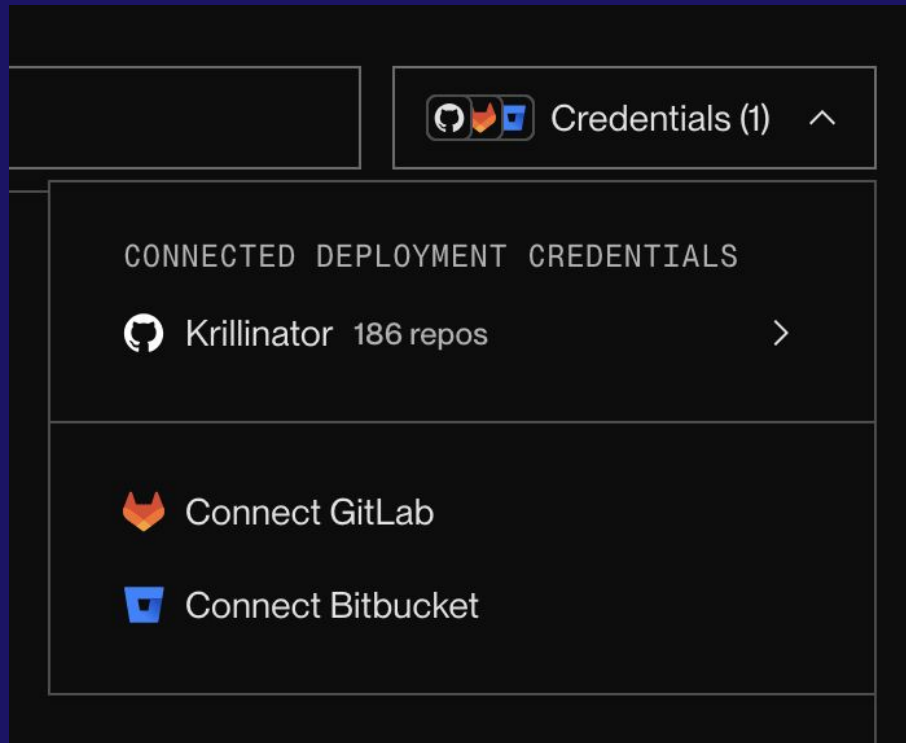
## Key Value

Managed Redis®-compatible storage. Ideal for use as a shared cache, message broker, or job queue.

[New Key Value Instance →](#)

# But First...

## (Install extension for GitHub)



Tryck först här

Det ska finnas ett val i listan här där du kan installera till github.

**Vi pausar här för att kolla tillsammans**



# New Project

(Pick your Project)

Git Provider

Public Git Repository

Existing Image

Search

 Credentials (1) ▾

 Krillinator /  WS\_node\_test 29m ago

View repo 

# New Project

## (What we got)

### Source Code

 Krillinator / WS\_node\_test • 29m ago

Edit

### Name

A unique name for your web service.

WS\_node\_test

### Project Optional

Add this web service to a project once it's created.



#### Create a new project to add this to?

You don't have any projects in this workspace. Projects allow you to group resources into environments so you can better manage related resources.

+ Create a project

Detta ser fint ut.. Vi behöver inte skapa något nytt "+ Create Project"

## Build Command

Render runs this command to build your app before each deploy.

```
$ npm install && npm run build
```

 Edit

## Pre-Deploy Command Optional

Render runs this command before the start command. Useful for database migrations and static asset uploads.

```
$
```



 Edit

## Start Command

Render runs this command to start your app with each deploy.

```
$ npm start
```

 Edit

Ändra följande så att det matchar allt ovan ^

# New Project (instance type)

## Instance Type

### For hobby projects

|                    |              |
|--------------------|--------------|
| <b>Free</b>        | 512 MB (RAM) |
| <b>\$0 / month</b> | 0.1 CPU      |

#### ⚠ Upgrade to enable more features

Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.

### For professional use

For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

- Zero Downtime
- SSH Access
- Scaling
- One-off jobs
- Support for persistent disks

|                    |              |
|--------------------|--------------|
| <b>Starter</b>     | 512 MB (RAM) |
| <b>\$7 / month</b> | 0.5 CPU      |

|                     |            |
|---------------------|------------|
| <b>Standard</b>     | 2 GB (RAM) |
| <b>\$25 / month</b> | 1 CPU      |

|                     |            |
|---------------------|------------|
| <b>Pro</b>          | 4 GB (RAM) |
| <b>\$85 / month</b> | 2 CPU      |

|                      |            |
|----------------------|------------|
| <b>Pro Plus</b>      | 8 GB (RAM) |
| <b>\$175 / month</b> | 4 CPU      |

|                      |             |
|----------------------|-------------|
| <b>Pro Max</b>       | 16 GB (RAM) |
| <b>\$225 / month</b> | 4 CPU       |

|                      |             |
|----------------------|-------------|
| <b>Pro Ultra</b>     | 32 GB (RAM) |
| <b>\$450 / month</b> | 8 CPU       |

Need a custom instance type? We support up to 512 GB RAM and 64 CPUs.

# New Project (instance type)

## Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

 Generate[+ Add Environment Variable](#)[Add from .env](#)[> Advanced](#)[Deploy Web Service](#)

**Notering:** Ibland kan lösenord, nycklar och 'secrets' behövas, då används **Environment Variables**.

Render listens on **port 10000** by default for web services.



M My Workspace 

 web\_Service\_Example\_Express

 Events

 Settings

MONITOR

 Logs

 Metrics

MANAGE

 Environment 

Environment 

+ Create environment group

Environment Variables 

Export 

Edit 

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

| KEY  | VALUE                                                                                     |
|------|-------------------------------------------------------------------------------------------|
| PORT | .....  |

Lägg till en environment som heter **'PORT'** och i Value tilldelar du **10000**

Done  
(Results)



<https://ws-node-test.onrender.com>

Hello world!



```
5  
6   app.get("/", (request, response) => {  
7     |   response.send("Hello world!")  
8   |   })  
9
```



# Question...

(Datatype?)



  <https://ws-node-test.onrender.com>

Hello world!

Vad är *"Hello World"* för datatype...?

**String? Json? XML? HTML?**

# What's happening?

## (Explanation)

```
app.get("/", (request, response) => {  
  response.send("Hello world!") // Automatically converts datatypes  
  response.json()               // Manually Converts to JSON  
})
```

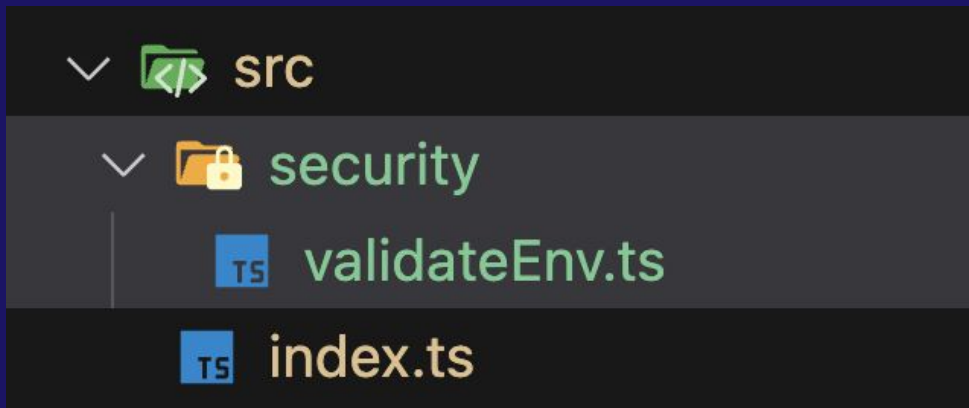
**Notera:** Om vi verkligen vill returnera JSON objekt explicit, skriv bara: `.json()`

# Separation of Concerns



# Separation of Concerns

(Secrets, keys, passwords...)



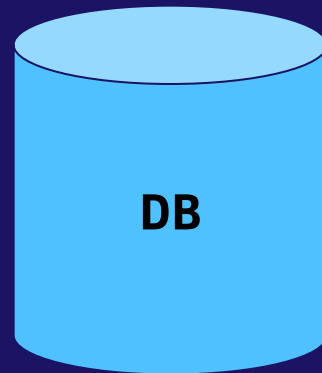
Flytta ut 'secret' och  
logik till 'security'

# Sammanfattning

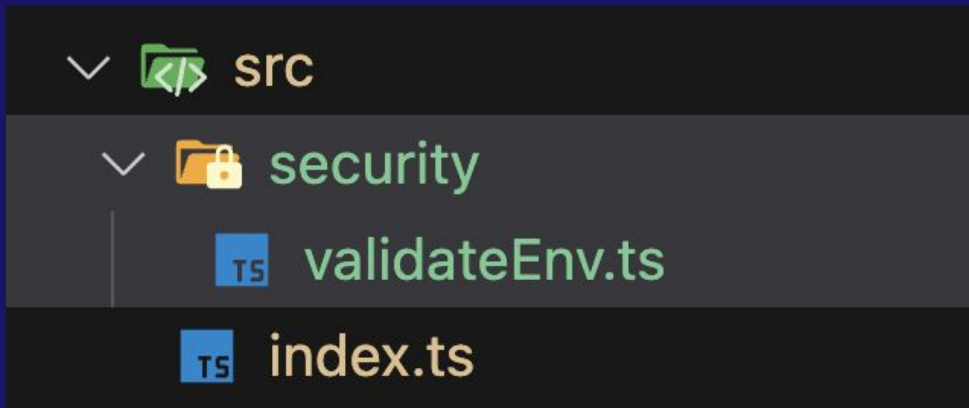
(Why did we do all of this?)

Det är viktigt att **få en grund** i hur hantering av nycklar, lösenord och övriga hemligheter på ett säkert sätt.

Detta då vi kommer **arbeta med databas nycklar** som andra kan få tag på



# validateEnv.ts Content (Validation)



Bör innehålla logik

```
const minSecretLength: number = 5

export function validateSecret (secret: string | undefined) {
  const typedSecret: string = validateType (secret)
  validateLength (typedSecret)

  return typedSecret
}

function validateType (secret: string | undefined): string {
  if (!secret) {
    throw new Error("Secret: not properly set up" )
  }

  return secret
}

function validateLength (secret: string) {
  if (secret && secret.length < minSecretLength) {
    throw new Error(`Secret: incorrect length (min length:
    ${minSecretLength})`)
  }
}
```

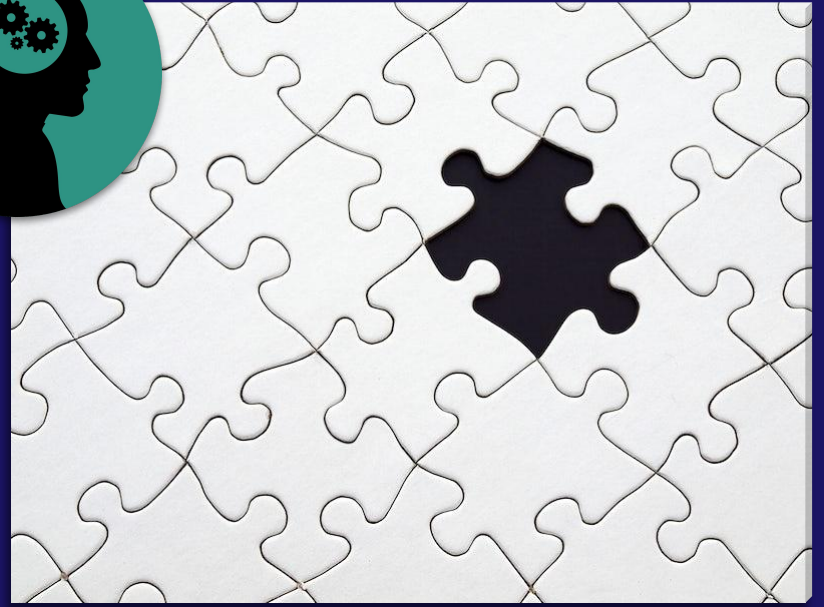
## Validation Logic (Bonus)

(validateEnv.ts)

Kod innehåller  
enkel validering  
och felhantering.



*Frågor?*







03

## Database Setup

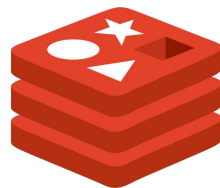
# Database of Choice

## In short



# Popular Choices

(But not limited to...)



redis

# MongoDB



## General purpose

- Don't know what you're building?  
*Awesome! Pick me!*
- Flexible JSON document model with dynamic schemas.
- Powerful query and aggregation capabilities.
- Disk-based, good for large datasets with sharding.

# Cassandra



## Smart data distribution

- Open Source
- Excels at massive, distributed write-heavy workloads.
- No single master node — fully peer-to-peer and fault tolerant.
- Adjustable consistency levels per operation.

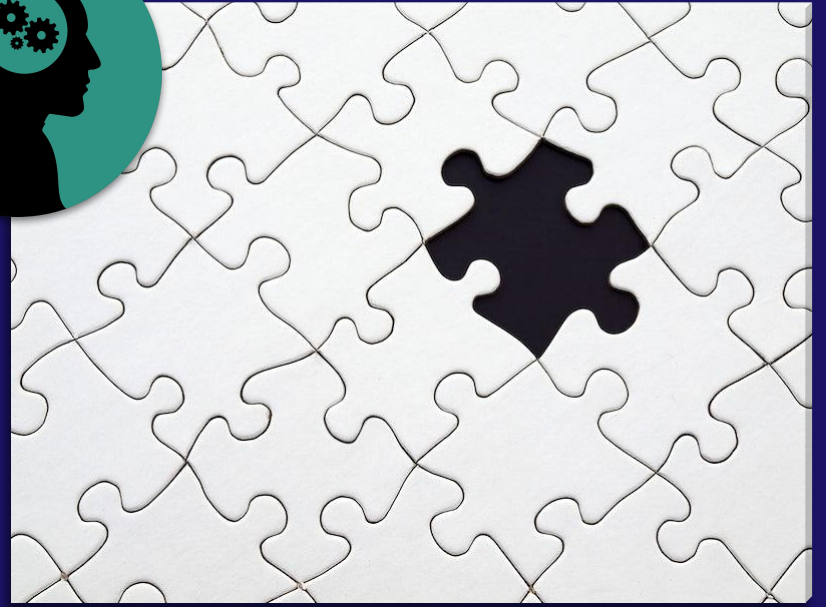
# Redis



## Caching Capabilities

- Caching, **no longer open source / PAID**
- In-memory for ultra-fast caching and real-time data.
- Supports advanced data structures and atomic ops.
- Built-in Pub/Sub and streaming features.

*Frågor?*



# Database of Choice

## MongoDB – noSQL





# Setup Account

## Work Lab



Navigera till  
<https://www.mongodb.com/>

Navigera till 'signin / signup'  
[https://account.mongodb.com/  
account/register](https://account.mongodb.com/account/register)

# Create Project MongoDB



# Projects

(Create New)

## ORGANIZATION

### Projects

Alerts 0

Activity Feed

Settings

Integrations

Access Manager

Resource Policies

Billing

Support

Live Migration

KRISTOFFER'S ORG - 2023-04-12

## Projects

Find a project...



Project Name

Clusters

Tags

Users

Teams

Alerts

Actions

New Project

# Name

## (WS\_2025)

### Name Your Project

Project names have to be unique within the organization (and other restrictions).

### Add Tags (Optional)

Use tags to efficiently label and categorize your projects. A project can have a maximum of 50 tags. You can modify tags for the project later. [Learn more](#)

| Key                                                         | Value | Actions                                                                                                                                           |
|-------------------------------------------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="text" value="Select a key or enter your own"/> | :     | <input type="text" value="Select a value or enter your own"/>  |
| <a href="#">+ Add tag</a>                                   |       |                                                                                                                                                   |
|                                                             |       | 0 TAGS                                                                                                                                            |

CancelNext

# Add Members / Permissions (Optional)

✓ Name Your Project

Add Members

## Add Members and Set Permissions

Invite new or existing users via email address...

Give your members access permissions below.

kristoffer.johansson@sti.se  
(you)

Project Owner

Back

Cancel

Create Project

# Cluster & DB

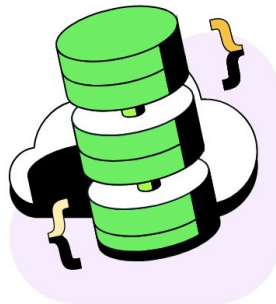
## MongoDB



# Create Cluster

(The Container of DB's)

## Overview



### Create a cluster

Choose your cloud provider, region, and specs.

+ Create

# Create Cluster (FREE TIER)

## Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.



**M10**

**\$0.08/hour**

Dedicated cluster for development environments and low-traffic applications.

| STORAGE | RAM  | vCPU    |
|---------|------|---------|
| 10 GB   | 2 GB | 2 vCPUs |

### Features

- ✓ Flexible backups ⓘ
- ✓ Zero-downtime cluster scaling ⓘ
- ✓ Performance diagnostic tools ⓘ
- ✓ Additional search indexes
- ✓ Uptime SLA ⓘ



**Flex**

**From \$0.011/hour**

Up to \$30/month

For development and testing, with on-demand burst capacity for unpredictable traffic.

| STORAGE | RAM    | vCPU   |
|---------|--------|--------|
| 5 GB    | Shared | Shared |

### Features

- ✓ Basic backups ⓘ
- ✓ Upgradeable ⓘ
- ✓ Pay As You Go pricing
- ✓ On-demand burst capacity



**Free**

For learning and exploring MongoDB in a cloud environment.

| STORAGE | RAM    | vCPU   |
|---------|--------|--------|
| 512 MB  | Shared | Shared |

### Features

- ✓ Free forever



# Config

## (changing name only)

### Configurations

#### Name

You cannot change the name once the cluster is created.

ws-2025

#### Provider



#### Region



★ Recommended ⓘ    🌿 Low carbon emissions ⓘ

#### Tag (optional)

Create your first tag to categorize and label your resources; more tags can be added later. [Learn more.](#)

Select or enter key

:

Select or enter value

### Quick setup

- ☒ Automate security setup ⓘ
- ☒ Preload sample dataset and create search index ⓘ

## Connect to ws-2025



### 2. Create a database user

This first user will have [atlasAdmin](#) permissions for this project.

We autogenerated a username and password. You can use this or create your own.

**i You'll need your database user's credentials in the next step. Copy the database user password.**

Username

Password

benny

123

HIDE

Copy

Create Database User

Close

Choose a connection method

**First user created = ADMIN**

**Viktigt att komma ihåg användarnamn & lösenord...**

## VIKTIGT

Du kan bara få åtkomst till din information från ditt **lokala** nätverk (där du befann dig när du skapat ditt kluster)

Under debugging kan det vara härligt att ändra platsåtkomst.

Publika nätverk kan blockera åtkomst (*bibliotek, caféer*)

### SECURITY

Quickstart

Backup

Database Access

**Network Access**

Advanced

Ändra till 0.0.0.0 eller lägg till en ny IP adress.

| IP Address                                   | Comment                                   | Status                | Actions                                     |
|----------------------------------------------|-------------------------------------------|-----------------------|---------------------------------------------|
| 0.0.0.0/0 (includes your current IP address) | Created as part of the Auto Setup process | <span>●</span> Active | <a href="#">EDIT</a> <a href="#">DELETE</a> |

✓ Sample dataset and search index successfully loaded. Get started with a sample query in [Data Explorer](#) or by connecting with the MongoDB Shell.

ws-2025

Connect

View Monitoring

Browse Collections

...

FREE

### Visualize Your Data

Build dashboards and charts, and embed them in your apps with MongoDB Charts.

Explore Charts

Dismiss

R 0

W

Last 6 minutes

100.0/s

i

Connections 1.0

Last 6 minutes

5.0

i

In 203.04 KB/s

Out 2.14 KB/s

Last 6 minutes

203.04 KB/s

i

Data Size

129.03 MB / 512.00 MB (25%)

Last 6 minutes

512.00 MB

VERSION

REGION

TYPE

BACKUPS

LINKED APP SERVICES

ATLAS SQL

ATLAS SEARCH

8.0.10

AWS / Stockholm (eu-north-1)

Replica Set - 3 nodes

Inactive

None Linked

[Connect](#)

[Create Index](#)

+ Add Tag

När allt är klart ska det se ut så här ^

✓ Sample dataset and search index successfully loaded. Get started with a sample query in [Data Explorer](#) or by connecting with the MongoDB Shell.



ws-2025

Connect

View Monitoring

Browse Collections



FREE

### Visualize Your Data

Build dashboards and charts, and embed them in your apps with MongoDB Charts.

Explore Charts

Dismiss

● R 0

● W

Last 6 minutes

100.0/s



Connections 1.0

Last 6 minutes

5.0



● In 203.04 KB/s

● Out 2.14 KB/s

Last 6 minutes

203.04 KB/s



Data Size

129.03 MB / 512.00 MB (25%)

Last 6 minutes

512.00 MB

Vi har fått data  
inbyggt, vi kikar!

# sample\_mflix.comments

STORAGE SIZE: 6.54MB

LOGICAL DATA SIZE: 11.14MB

TOTAL DOCUMENTS: 41079

INDEXES TOTAL SIZE: 1.43MB

Find

Indexes

Schema Anti-Patterns 0

Aggregation

Search Indexes

[Generate queries from natural language in Compass](#)

INSERT DOCUMENT

Filter

{}

Reset

Apply

Options ▶

QUERY RESULTS: 1-20 OF MANY



```
_id: ObjectId('5a9427648b0beeb69579e7')
name: "Mercedes Tyler"
email: "mercedes_tyler@fakegmail.com"
movie_id: ObjectId('573a1390f29313caabcd4323')
text: "Eius veritatis vero facilis quaerat fuga temporibus. Praesentium exped..."
date: 2002-08-18T04:56:07.000+00:00
```



**Vi har objekt med:** Namn, email, movie\_id, text och datum!

*Men vi kan inte se dessa..? NEXT PAGE*

## ▼ sample\_mflix

comments

embedded\_movies

movies

sessions

theaters

users

Alla dessa flikar...  
Är egentligen tabeller!

Men MongoDB arbetar med något  
som kallas ***dokument***

***Därför ser det lite annorlunda  
ut!***

# noSQL & MongoDB Datatypes





|   |                                                                                              |          |
|---|----------------------------------------------------------------------------------------------|----------|
| 1 | <code>_id: ObjectId('5a9427648b0beeb69579e7')</code>                                         | ObjectId |
| 2 | <code>name : "Mercedes Tyler"</code>                                                         | String   |
| 3 | <code>email : "mercedes_tyler@fakegmail.com"</code>                                          | String   |
| 4 | <code>movie_id : 573a1390f29313caabcd4323</code>                                             | ObjectId |
| 5 | <code>text : "Eius veritatis vero facilis quaerat fuga temporibus. Praesentium exped"</code> | String   |
| 6 | <code>date : 2002-08-18T04:56:07.000+00:00</code>                                            | Date     |

**Läs mer:** <https://www.mongodb.com/docs/manual/reference/bson-types/>

**Vi kan utesluta att det finns många  
bekanta och även nya datatyper!**

The screenshot shows the MongoDB Compass web interface. On the left sidebar, the 'sample\_mflix' database is selected, and the 'embedded\_movies' collection is highlighted with a blue arrow. The main panel displays a search filter: `{ "awards.wins": { "$gt": 0 } }`, also indicated by a blue arrow. To the right of the filter are buttons for 'Reset', 'Apply', and 'Options'. Below the filter, a document is shown with the following structure:

```
fullplot: "Michael "Beau" Geste leaves England in disgrace and joins the infamous..."
languages: Array (1)
released: 1926-08-25T00:00:00.000+00:00
directors: Array (1)
writers: Array (4)
awards: Object
  wins: 1
  nominations: 0
  text: "1 win."
lastupdated: "2015-08-25 00:40:54.383000000"
year: 1926
imdb: Object
```

At the bottom of the interface, there are navigation buttons: '< PREVIOUS', '1-20 of many results', and 'NEXT >'. The 'NEXT >' button is highlighted with a green background.

```
{ "awards.wins": { "$gt": 0 } }
```

Syntax översätts som: Hitta awards, gå in i awards och leta efter wins objekt.  
Gt = greater than

# Node App & MongoDB Installation & Connection



## Overview

## DATABASE

Clusters

## SERVICES

Atlas Search

Stream Processing

Triggers

Migration

Data Federation

## SECURITY

Quickstart

Backup

KRISTOFFER'S ORG - 2023-04-12 &gt; WS\_2025

## Overview

## Clusters

Create cluster



ws-2025

Connect

Edit configuration

Data Size: 135.4 MB



Browse collections



View monitoring



+ Add Tag

# Connecting with MongoDB Driver

## 1. Select your driver and version

We recommend installing and using the latest driver version.

Driver

Version

Node.js

6.7 or later

## 2. Install your driver

Run the following on the command line

```
npm install mongodb
```



[View MongoDB Node.js Driver installation instructions.](#)

## 3. Add your connection string into your application code

Use this connection string in your application

☐ View full code sample

```
mongodb+srv://Benny:<db_password>@ws-2025.7geqiko.mongodb.net/?  
retryWrites=true&w=majority&appName=ws-2025
```



Replace **<db\_password>** with the password for the **Benny** database user. Ensure any option params are [URL encoded](#).

Install first

Copy paste this!

🔑 .env

```
1 MY_GLOBAL_TEST_SECRET="g4Z6RMnksmc0GNYpnWh52SBe+3klD1g991B
2 DB_CONNECTION_STRING="mongodb+srv://Benny:123@ws-2025.7geq
3 DB_NAME="sample_mflix"
4
```

**Notera:** 'DB\_CONNECTION\_STRING' är ett variabel namn

**Viktigt:** Detta måste vara hemligt och ska inte upp på Github

# Databas uppsättning

## (database.ts)



Ansvarsområden:

- Skapande av '**Mongo-client**'
- **Öppnar** databasanslutning
- **Stänger** databasanslutning
- '**Getter**' för databasnamn

# Database.ts

## (Part #1 – Client Setup)

```
import { Db, MongoClient, ServerApiVersion } from "mongodb"
import { validateSecret } from "../security/validateEnv"

// Runtime-Validation
const uri: string = validateSecret(process.env.DB_CONNECTION_STRING)
const dbName: string = validateSecret(process.env.DB_NAME)
const client = new MongoClient(uri, {
  serverApi: {
    version: ServerApiVersion.v1,
    strict: true,
    deprecationErrors: true,
  },
})

let isConnected = false
```



# Database.ts

## (Part #2 – Establish Connection)

```
/** Attempts to connect to database.  
 * Returns a promise.  
 * Throws error at failed attempt. */  
export async function runDB(): Promise<void> {  
  if (isConnected) return // prevent duplicate connects  
  
  try {  
    await client.connect()  
    await client.db("admin").command({ ping: 1 })  
    isConnected = true  
    console.log("Db is up and running")  
  } catch (err) {  
    console.error(err)  
    throw err  
  }  
}
```

# Database.ts

## (Part #3 – Helper Methods)

```
export function getDB(): Db {  
  if (!isConnected) {  
    throw new Error(" Tried to access DB before connecting")  
  }  
  return client.db(dbName)  
}  
  
export async function closeDB(): Promise<void> {  
  await client.close()  
  isConnected = false  
  console.log("Mongodb connection closed")  
}
```

# index.ts

## (Part #4 – Run)

```
import express from "express"
import "dotenv/config" // oneliner for configuration
import { closeDB, runDB } from "../db/database"

async function startServer() {
  try {
    await runDB()
    app.listen(port, () => {
      console.log(`Listening to port ${port}`)
      console.log(`Start the app: http://localhost:${port}`)
    })
    process.on("SIGINT", async () => {
      console.log("Cleaning up...")
      await closeDB()
      process.exit(0)
    })
  } catch (error) {
    console.log(error)
  }
}

startServer()
```

# Working With Requests Basics



# Requests – Basics!

## (Hello World – HTML)

```
app.get("/", (request, response) => {  
  response.send("Hello world!")  
})
```



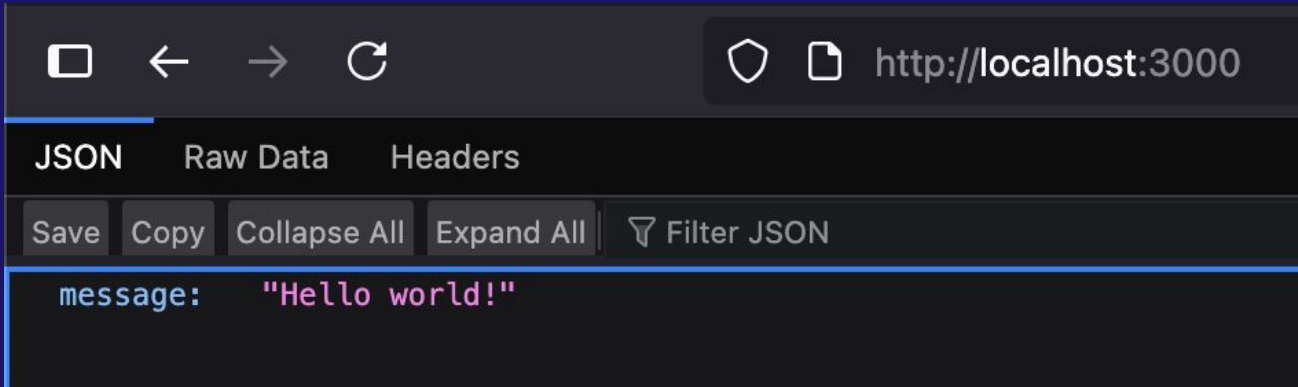
http://localhost:3000

Hello world!

# Requests – Basics!

## (Hello World – JSON)

```
app.get("/", (request, response) => {  
  response.send({ message: "Hello world!" })  
})
```



# Requests – Basics!

## (Status Codes)

```
app.get("/", (request, response) => {  
  response.status(200).send({ message: "Hello world!" })  
})
```

# HTTP Status Codes



1XX  
INFORMATIONAL

2XX  
SUCCESS

3XX  
REDIRECTION

4XX  
CLIENT ERROR

5XX  
SERVER ERROR



# Dynamic Params Basics

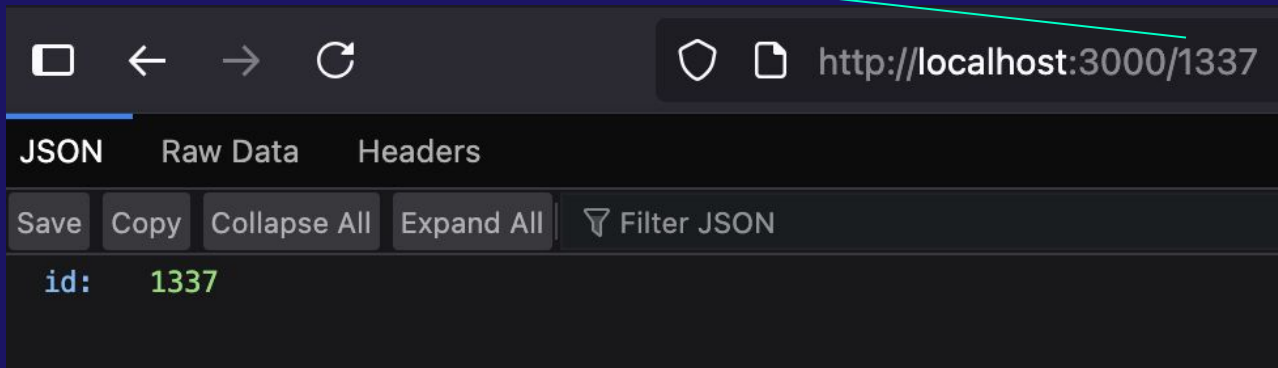


# Dynamic Parameters (Request Params)

```
app.get("/:id", (req: Request, res: Response) => {  
  const id = req.params.id  
  
  res.send({ id: id })  
})
```

# Dynamic Parameters (Request Params)

```
app.get("/:id", (req: Request, res: Response) => {})
```



# Dynamic Parameters

## (Error Handling)

```
app.get("/:id", (req: Request, res: Response) => {  
  const id: number = Number(req.params.id) // CASTING  
  
  if (isNaN(id)) {  
    res.status(400).send("Not a number")  
    return  
  }  
  
  res.send({ id: id })  
})
```

# Return void (Multiple responses)

```
app.get("/:id", (req: Request, res: Response) => {  
  const id: number = Number(req.params.id) // CASTING  
  
  if (isNaN(id)) {  
    res.status(400).send("Not a number")  
    return  
  }  
  
  res.send({ id: id })  
})
```



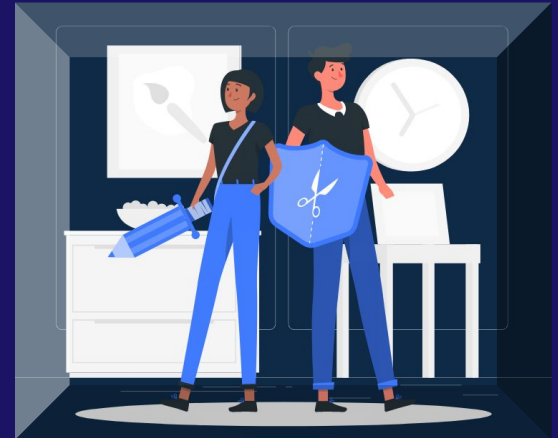
04

Uppgifter  
&  
Eget Arbete

## Välkommen till Uppgifterna!

Uppgifterna är till för att testa dina färdigheter och kunskaper för att både öva och repetera på det vi har arbetat med under föreläsningarna.

## Uppgifter



# MINNS DU?

```
// Vad är ett 'cluster' inom MongoDB?  
// Räknas 'cluster' som databas?  
// Varför, varför inte?  
  
// Vad står NoSQL för?
```



mongoDB



# FACIT

## Vad är ett 'cluster' inom MongoDB?

- Ett *cluster* i MongoDB är en samling av servrar (noder) som samarbetar för att lagra och hantera data, ofta för att möjliggöra hög tillgänglighet, lastbalansering och skalbarhet

## Räknas 'cluster' som databas? Varför, varför inte?

- Nej, ett *cluster* räknas inte som en databas. Ett cluster är infrastrukturen (servermiljön) där en eller flera databaser kan finnas. Själva databasen är en logisk enhet för datalagring, medan ett cluster är den fysiska/tekniska miljön som kan innehålla många databaser

## Vad står NoSQL för?

- NoSQL står för "Not Only SQL" och syftar på databaser som inte är strikt relationsbaserade. De är utformade för att hantera stora mängder ostrukturerad eller semi-strukturerad data och använder andra lagringsmodeller än tabeller och rader, t.ex. dokument, nyckel-värde, kolumner eller grafer

# MINNS DU?

```
// Vad är en 'Connection String'?  
// Hur hänger detta ihop med .env filer?  
  
/*  
Vad är skillnaden på en 'lokal' app och  
'produktion'  
*/
```



# FACIT

En **Connection String** är en textsträng som innehåller information för att koppla upp sig mot en databas, t.ex. användarnamn, lösenord, databasadress och port.

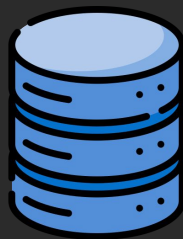
Den sparas ofta i en **.env-fil** för att hålla känslig information utanför koden och göra det lättare att byta miljö.

Skillnaden mellan en **lokal** app och **produktion** är att den lokala körs på din egen dator för utveckling och test, medan produktionen är den färdiga versionen som körs "live" för riktiga användare.



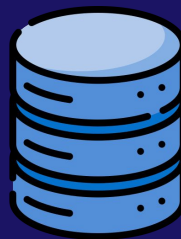
# RÄTT ELLER FEL

1. **await** gör så att koden väntar på databasens svar innan den går vidare.
2. Du behöver inte använda **await** för asynkrona databasfrågor – det spelar ingen roll.
3. **await** kan bara användas inuti funktioner som är märkta med **async**.



# FACIT

1. **await** gör så att koden väntar på databasens svar innan den går vidare.  
**TRUE**
2. Du behöver inte använda **await** för asynkrona databasfrågor – det spelar ingen roll.  
**FALSE** *(utan await får du inte tillbaka resultatet direkt, vilket kan orsaka buggar)*
3. **await** kan bara användas inuti funktioner som är märkta med **async**.  
**TRUE**



# KOD ANALYS

```
async function startServer() {  
  try {  
    await runDB()  
    app.listen(port, () => {  
      console.log(`Listening to port ${port}`)  
      console.log(`Start the app: http://localhost:${port}`)  
    })  
    process.on("SIGINT", async () => {  
      console.log("Cleaning up...")  
      await closeDB()  
      process.exit(0)  
    })  
  } catch (error) {  
    console.log(error)  
  }  
}
```

*Kan du förklara  
varför kodens olika  
steg?  
Varför är det  
upplagt så?*

# KOD ANALYS – FACIT

Koden är strukturerad så här för att  
se till att:

- #1. databasen är igång innan servern startar
- #2. hantera oväntade fel smidigt och
- #3. stänga ner allt korrekt när appen avslutas.

```
1 // -Uppgift #1- //
```

```
2
```

```
3 /* INSTRUCTIONS
```

```
4
```

```
5     Öppna Projekt: WS_Uppgifter_node
```

```
6     Navigera till din branch
```

```
7         $ git branch -a (listar alla grenar)
```

```
8         $ git checkout 'branchName' (navigerar)
```

```
9
```

```
10     Installera (Stå inom projekt i terminal)
```

```
11     $ npm install mongodb
```

```
12
```

```
13
```

```
14     Skapa konto (om ni inte har ett)
```

```
15     Render: https://dashboard.render.com/register
```

```
16     Mongo: https://account.mongodb.com/account/register
```

```
17
```

```
18     Inom mongoDB, skapa ett 'Cluster' med
```

```
19     fördefinierad data.
```

```
20
```

```
21     Hämta 'Connection URL'
```

```
22
```

```
23 */
```

# Uppgift #1



```

1          // -Uppgift #2- //
2
3      /* INSTRUCTIONS
4
5      // Runtime-Validation
6      const uri: string | undefined = process.env.DB_CONNECTION_STRING
7      const dbName: string | undefined = process.env.DB_NAME
8      const client = new MongoClient(uri, {
9          serverApi: {
10              version: ServerApiVersion.v1,
11              strict: true,
12              deprecationErrors: true,
13          },
14      })
15
16      let isConnected = false
17
18
19      */
20
21      // HINT & Examples
22      hint("Notera att detta är ett ENKLARE upplägg")
23

```

## Uppgift #2

Detta kod stycke skrivs  
inom: *database.ts*

```
1 // -Uppgift #3- //
2 Theory
3 /* INSTRUCTIONS
4
5 Notera följande kod
6 app.listen(port, () => {
7   console.log(`Listening to port ${port}`)
8   console.log(`Start the app: http://localhost:${port}`)
9 })
```

**Fråga:** När startar vi databasen, innan eller efter app.listen?

```
14 */
```

```
15 // HINT & Examples
```

```
16 hint(" Detta är en viktig teoretisk fråga som hjälper
17 oss definiera logik inom rätt ordning.
```

```
18
19 Om vi exekverar databas körning efter att något har gått
20 sönder riskerar vi flertal problem... ")
```

```
21
22 hint(" Vad händer om databasen inte kan uppkopplas?")
23
```

## Uppgift #3

*Facit på nästa sida!*

```
1
2
3 // FACIT
4 solution("
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
```

Vi startar **databasen innan** `app.listen()` körs.

**Motivering:**

Om vi startar servern **innan** databasen är igång, kan appen börja ta emot förfrågningar trots att den inte kan hämta eller spara data.. vilket leder till fel, brutna endpoints och förvirrade användare.

Genom att invänta att databasen är korrekt uppkopplad först, garanterar vi att appen är redo att hantera inkommande trafik utan problem.

)

```
1           // -Uppgift #4- //
2           Theory
3  /* INSTRUCTIONS
4
5      app.get("/:id", (req: Request, res: Response) => {})
6
7      Vad exakt är en dynamisk parameter?
8      Hur urskiljer sig req från res?
9
10  */
11
12  // HINT & Examples
13  hint("Vad står req och res för?")
14
15
16
17
18
19
20
21
22
23
```

## Uppgift #4

```

1          // -Uppgift #5- //
2              Theory
3  /* INSTRUCTIONS
4
5      Om vi väljer att skicka flertal response's,
6      Vad händer när vi går in på: localhost:3000/2
7
8      app.get("/:id", (req: Request, res: Response) => {
9          const id: number = Number(req.params.id) // CASTING
10
11          if (isNaN(id)) {
12              res.status(400).send("Not a number")
13          }
14
15          res.send({ id: id })
16      })
17
18  */
19
20
21  // HINT & Examples
22  hint("Notera saknad av 'return' ")
23

```

## Uppgift #5

```
1          // -Uppgift #6- //
2          Versionshantering
3  /* INSTRUCTIONS
4
5      Skapa en enkel 'endpoint'
6      Definiera interface User
7
8      Definiera en User Variabel som skickas med i
9      ett Response objekt med status koden 201 (created)
10
11     Testa och kör koden
12     Tillför sedan en commit. Du är nu klar!
13
14  /* HINT & Examples
15  hint("
16      res.send()
17      res.status()
18      Definiera först status innan send()
19  ")
20
21
22
23
```

## Uppgift #6

# THANKS!

Do you have any questions?  
kristoffer.johansson@sti.se

[sti.learning.nu/](https://sti.learning.nu/)

CREDITS: This presentation template was created by  
Slidesgo, including icons by Flaticon, and  
infographics & images by Freepik.

*You can also contact me VIA Teams (quicker response)*  
*Du kan också kontakta mig VIA Teams (Snabbare svar)*